EA-0060 - AQELYN Engineering Archive

EA-0060 — AI Engineering & Prompt Handbook

Brand: AQELYN

Subtitle: AI prompting, retrieval, explainability, safety, evaluation, and governance standard

Status: FULL_COMPLETE

Date: 2026-07-09

Executive Summary

Define the AI engineering rules that make AQELYN recommendations evidence-based, explainable, safe, auditable, and consistent.

This document is part of the AQELYN / AQELYN Architecture Baseline v1.0 and is normative for implementation. It supports the product principles: Explain Before You Recommend, Simplicity First, Evidence Before Opinion, Human-Centered Security, and Expert Depth on Demand.

Scope Domains

• Prompt templates

• RAG and knowledge graph grounding

• Evidence-based explanations

• Risk scoring narratives

• Human-centered security responses

• Guardrails and safety

• Evaluation and observability

• Prompt version control

01. AI Mission

This section defines the mandatory rules, constraints, and implementation expectations for **AI Mission** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-01-01:** The implementation shall apply ai mission consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-01-02:** The implementation shall apply ai mission consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-01-03:** The implementation shall apply ai mission consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-01-04:** The implementation shall apply ai mission consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-01-05:** The implementation shall apply ai mission consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: AI Mission is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: AI Mission is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: AI Mission is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: AI Mission is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to AI Mission, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

02. AI Safety Principles

This section defines the mandatory rules, constraints, and implementation expectations for **AI Safety Principles** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-02-01:** The implementation shall apply ai safety principles consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-02-02:** The implementation shall apply ai safety principles consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-02-03:** The implementation shall apply ai safety principles consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-02-04:** The implementation shall apply ai safety principles consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-02-05:** The implementation shall apply ai safety principles consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: AI Safety Principles is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: AI Safety Principles is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: AI Safety Principles is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: AI Safety Principles is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to AI Safety Principles, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

03. Prompt Taxonomy

This section defines the mandatory rules, constraints, and implementation expectations for **Prompt Taxonomy** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-03-01:** The implementation shall apply prompt taxonomy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-03-02:** The implementation shall apply prompt taxonomy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-03-03:** The implementation shall apply prompt taxonomy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-03-04:** The implementation shall apply prompt taxonomy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-03-05:** The implementation shall apply prompt taxonomy consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Prompt Taxonomy is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Prompt Taxonomy is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Prompt Taxonomy is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Prompt Taxonomy is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Prompt Taxonomy, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

04. System Prompt Standards

This section defines the mandatory rules, constraints, and implementation expectations for **System Prompt Standards** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-04-01:** The implementation shall apply system prompt standards consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-04-02:** The implementation shall apply system prompt standards consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-04-03:** The implementation shall apply system prompt standards consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-04-04:** The implementation shall apply system prompt standards consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-04-05:** The implementation shall apply system prompt standards consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: System Prompt Standards is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: System Prompt Standards is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: System Prompt Standards is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: System Prompt Standards is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to System Prompt Standards, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

05. Finding Explanation Prompts

This section defines the mandatory rules, constraints, and implementation expectations for **Finding Explanation Prompts** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-05-01:** The implementation shall apply finding explanation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-05-02:** The implementation shall apply finding explanation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-05-03:** The implementation shall apply finding explanation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-05-04:** The implementation shall apply finding explanation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-05-05:** The implementation shall apply finding explanation prompts consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Finding Explanation Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Finding Explanation Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Finding Explanation Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Finding Explanation Prompts is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Finding Explanation Prompts, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

06. Remediation Prompts

This section defines the mandatory rules, constraints, and implementation expectations for **Remediation Prompts** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-06-01:** The implementation shall apply remediation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-06-02:** The implementation shall apply remediation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-06-03:** The implementation shall apply remediation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-06-04:** The implementation shall apply remediation prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-06-05:** The implementation shall apply remediation prompts consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Remediation Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Remediation Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Remediation Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Remediation Prompts is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Remediation Prompts, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

07. Risk Summary Prompts

This section defines the mandatory rules, constraints, and implementation expectations for **Risk Summary Prompts** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-07-01:** The implementation shall apply risk summary prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-07-02:** The implementation shall apply risk summary prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-07-03:** The implementation shall apply risk summary prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-07-04:** The implementation shall apply risk summary prompts consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-07-05:** The implementation shall apply risk summary prompts consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Risk Summary Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Risk Summary Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Risk Summary Prompts is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Risk Summary Prompts is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Risk Summary Prompts, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

08. Evidence Grounding

This section defines the mandatory rules, constraints, and implementation expectations for **Evidence Grounding** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-08-01:** The implementation shall apply evidence grounding consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-08-02:** The implementation shall apply evidence grounding consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-08-03:** The implementation shall apply evidence grounding consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-08-04:** The implementation shall apply evidence grounding consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-08-05:** The implementation shall apply evidence grounding consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Evidence Grounding is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Evidence Grounding is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Evidence Grounding is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Evidence Grounding is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Evidence Grounding, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

09. RAG Architecture

This section defines the mandatory rules, constraints, and implementation expectations for **RAG Architecture** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-09-01:** The implementation shall apply rag architecture consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-09-02:** The implementation shall apply rag architecture consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-09-03:** The implementation shall apply rag architecture consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-09-04:** The implementation shall apply rag architecture consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-09-05:** The implementation shall apply rag architecture consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: RAG Architecture is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: RAG Architecture is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: RAG Architecture is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: RAG Architecture is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to RAG Architecture, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

10. Knowledge Graph Integration

This section defines the mandatory rules, constraints, and implementation expectations for **Knowledge Graph Integration** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-10-01:** The implementation shall apply knowledge graph integration consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-10-02:** The implementation shall apply knowledge graph integration consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-10-03:** The implementation shall apply knowledge graph integration consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-10-04:** The implementation shall apply knowledge graph integration consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-10-05:** The implementation shall apply knowledge graph integration consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Knowledge Graph Integration is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Knowledge Graph Integration is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Knowledge Graph Integration is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Knowledge Graph Integration is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Knowledge Graph Integration, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

11. Confidence Scoring

This section defines the mandatory rules, constraints, and implementation expectations for **Confidence Scoring** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-11-01:** The implementation shall apply confidence scoring consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-11-02:** The implementation shall apply confidence scoring consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-11-03:** The implementation shall apply confidence scoring consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-11-04:** The implementation shall apply confidence scoring consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-11-05:** The implementation shall apply confidence scoring consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Confidence Scoring is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Confidence Scoring is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Confidence Scoring is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Confidence Scoring is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Confidence Scoring, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

12. Hallucination Mitigation

This section defines the mandatory rules, constraints, and implementation expectations for **Hallucination Mitigation** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-12-01:** The implementation shall apply hallucination mitigation consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-12-02:** The implementation shall apply hallucination mitigation consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-12-03:** The implementation shall apply hallucination mitigation consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-12-04:** The implementation shall apply hallucination mitigation consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-12-05:** The implementation shall apply hallucination mitigation consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Hallucination Mitigation is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Hallucination Mitigation is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Hallucination Mitigation is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Hallucination Mitigation is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Hallucination Mitigation, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

13. AI Memory Policy

This section defines the mandatory rules, constraints, and implementation expectations for **AI Memory Policy** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-13-01:** The implementation shall apply ai memory policy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-13-02:** The implementation shall apply ai memory policy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-13-03:** The implementation shall apply ai memory policy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-13-04:** The implementation shall apply ai memory policy consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-13-05:** The implementation shall apply ai memory policy consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: AI Memory Policy is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: AI Memory Policy is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: AI Memory Policy is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: AI Memory Policy is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to AI Memory Policy, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

14. Multi-Agent Coordination

This section defines the mandatory rules, constraints, and implementation expectations for **Multi-Agent Coordination** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-14-01:** The implementation shall apply multi-agent coordination consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-14-02:** The implementation shall apply multi-agent coordination consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-14-03:** The implementation shall apply multi-agent coordination consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-14-04:** The implementation shall apply multi-agent coordination consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-14-05:** The implementation shall apply multi-agent coordination consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Multi-Agent Coordination is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Multi-Agent Coordination is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Multi-Agent Coordination is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Multi-Agent Coordination is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Multi-Agent Coordination, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

15. AI Observability

This section defines the mandatory rules, constraints, and implementation expectations for **AI Observability** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-15-01:** The implementation shall apply ai observability consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-15-02:** The implementation shall apply ai observability consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-15-03:** The implementation shall apply ai observability consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-15-04:** The implementation shall apply ai observability consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-15-05:** The implementation shall apply ai observability consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: AI Observability is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: AI Observability is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: AI Observability is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: AI Observability is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to AI Observability, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

16. Evaluation Harness

This section defines the mandatory rules, constraints, and implementation expectations for **Evaluation Harness** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-16-01:** The implementation shall apply evaluation harness consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-16-02:** The implementation shall apply evaluation harness consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-16-03:** The implementation shall apply evaluation harness consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-16-04:** The implementation shall apply evaluation harness consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-16-05:** The implementation shall apply evaluation harness consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Evaluation Harness is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Evaluation Harness is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Evaluation Harness is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Evaluation Harness is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Evaluation Harness, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

17. Red Team Testing

This section defines the mandatory rules, constraints, and implementation expectations for **Red Team Testing** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-17-01:** The implementation shall apply red team testing consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-17-02:** The implementation shall apply red team testing consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-17-03:** The implementation shall apply red team testing consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-17-04:** The implementation shall apply red team testing consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-17-05:** The implementation shall apply red team testing consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Red Team Testing is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Red Team Testing is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Red Team Testing is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Red Team Testing is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Red Team Testing, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

18. Prompt Versioning

This section defines the mandatory rules, constraints, and implementation expectations for **Prompt Versioning** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-18-01:** The implementation shall apply prompt versioning consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-18-02:** The implementation shall apply prompt versioning consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-18-03:** The implementation shall apply prompt versioning consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-18-04:** The implementation shall apply prompt versioning consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-18-05:** The implementation shall apply prompt versioning consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: Prompt Versioning is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: Prompt Versioning is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: Prompt Versioning is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: Prompt Versioning is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to Prompt Versioning, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

19. User-Level Communication

This section defines the mandatory rules, constraints, and implementation expectations for **User-Level Communication** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-19-01:** The implementation shall apply user-level communication consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-19-02:** The implementation shall apply user-level communication consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-19-03:** The implementation shall apply user-level communication consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-19-04:** The implementation shall apply user-level communication consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-19-05:** The implementation shall apply user-level communication consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: User-Level Communication is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: User-Level Communication is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: User-Level Communication is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: User-Level Communication is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to User-Level Communication, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

20. AI Definition of Done

This section defines the mandatory rules, constraints, and implementation expectations for **AI Definition of Done** within the AQELYN Platform. The guidance is designed to be directly actionable by Codex and human engineers.

Requirements

• **EA-0060-REQ-20-01:** The implementation shall apply ai definition of done consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-20-02:** The implementation shall apply ai definition of done consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-20-03:** The implementation shall apply ai definition of done consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-20-04:** The implementation shall apply ai definition of done consistently across all affected modules and maintain traceability to source requirements.

• **EA-0060-REQ-20-05:** The implementation shall apply ai definition of done consistently across all affected modules and maintain traceability to source requirements.

Implementation Rules

• All code and configuration must be deterministic, reviewable, and testable.

• All external inputs must be validated before processing.

• All user-facing outputs must be understandable by non-experts and expandable for experts.

• All security decisions must include evidence references and audit metadata.

• All deviations require an Architecture Decision Record and engineering review.

Acceptance Criteria

• Acceptance item 1: AI Definition of Done is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 2: AI Definition of Done is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 3: AI Definition of Done is implemented, tested, documented, and linked to the traceability matrix.

• Acceptance item 4: AI Definition of Done is implemented, tested, documented, and linked to the traceability matrix.

Codex Implementation Notes

Codex shall treat this section as binding. When implementing features related to AI Definition of Done, it shall generate unit tests, integration tests where applicable, and update documentation artifacts in the same pull request.

Requirements Matrix

Requirement ID	Requirement	Priority	Verification
EA-0060-REQ-01	Implement AI Mission controls	Mandatory	Review + Tests
EA-0060-REQ-02	Implement AI Safety Principles controls	Mandatory	Review + Tests
EA-0060-REQ-03	Implement Prompt Taxonomy controls	Mandatory	Review + Tests
EA-0060-REQ-04	Implement System Prompt Standards controls	Mandatory	Review + Tests
EA-0060-REQ-05	Implement Finding Explanation Prompts controls	Mandatory	Review + Tests
EA-0060-REQ-06	Implement Remediation Prompts controls	Mandatory	Review + Tests
EA-0060-REQ-07	Implement Risk Summary Prompts controls	Mandatory	Review + Tests
EA-0060-REQ-08	Implement Evidence Grounding controls	Mandatory	Review + Tests
EA-0060-REQ-09	Implement RAG Architecture controls	Mandatory	Review + Tests
EA-0060-REQ-10	Implement Knowledge Graph Integration controls	Mandatory	Review + Tests
EA-0060-REQ-11	Implement Confidence Scoring controls	Mandatory	Review + Tests
EA-0060-REQ-12	Implement Hallucination Mitigation controls	Mandatory	Review + Tests
EA-0060-REQ-13	Implement AI Memory Policy controls	Mandatory	Review + Tests
EA-0060-REQ-14	Implement Multi-Agent Coordination controls	Mandatory	Review + Tests
EA-0060-REQ-15	Implement AI Observability controls	Mandatory	Review + Tests
EA-0060-REQ-16	Implement Evaluation Harness controls	Mandatory	Review + Tests
EA-0060-REQ-17	Implement Red Team Testing controls	Mandatory	Review + Tests
EA-0060-REQ-18	Implement Prompt Versioning controls	Mandatory	Review + Tests
EA-0060-REQ-19	Implement User-Level Communication controls	Mandatory	Review + Tests
EA-0060-REQ-20	Implement AI Definition of Done controls	Mandatory	Review + Tests

Traceability Matrix

Requirement	Source	Implementation Target	Verification Evidence
EA-0060-REQ-01	EA-0060 Section 01	src/, tests/, api/, sdk/ as applicable	CI logs, unit tests, review checklist
EA-0060-REQ-02	EA-0060 Section 02	src/, tests/, api/, sdk/ as applicable	CI logs, unit tests, review checklist
EA-0060-REQ-03	EA-0060 Section 03	src/, tests/, api/, sdk/ as applicable	CI logs, unit tests, review checklist

| EA-0060-REQ-04 | EA-0060 Section 04 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-05 | EA-0060 Section 05 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-06 | EA-0060 Section 06 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-07 | EA-0060 Section 07 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-08 | EA-0060 Section 08 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-09 | EA-0060 Section 09 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-10 | EA-0060 Section 10 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-11 | EA-0060 Section 11 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-12 | EA-0060 Section 12 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-13 | EA-0060 Section 13 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-14 | EA-0060 Section 14 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-15 | EA-0060 Section 15 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-16 | EA-0060 Section 16 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-17 | EA-0060 Section 17 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-18 | EA-0060 Section 18 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-19 | EA-0060 Section 19 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

| EA-0060-REQ-20 | EA-0060 Section 20 | src/, tests/, api/, sdk/ as applicable | CI logs, unit tests, review checklist |

Engineering Journal

• 2026-07-09: EA-0060 created as part of the final architecture completion set EA-0058 through EA-0061.

• Decision: AQELYN branding is applied while AQELYN remains the internal engineering codename.

• Decision: this package is normative for coding readiness and Codex execution.

Final Review

The package is approved for implementation baseline use when all files are present, hashes are generated, diagrams render, and PDF/HTML outputs correspond to the Master Markdown.